GUÍA

ARCHITECTURE AGENT
EN SPEC-DRIVEN DEVELOPMENT

De la SPEC aprobada a una arquitectura técnica justificable

Guía de trabajo para equipos de ingeniería de software con Inteligencia Artificial

Versión 1.0 · 2026

1. Propósito de la guía

Esta guía define el papel del Architecture Agent dentro de un proceso de Spec-Driven Development. El agente recibe una SPEC aprobada y transforma sus requisitos, restricciones y atributos de calidad en una propuesta de arquitectura técnica coherente, justificable, documentada y trazable.

Pregunta central: "¿Cómo debemos estructurar técnicamente el sistema para satisfacer la SPEC y sus atributos de calidad?"

2. Ubicación dentro del flujo Spec-Driven

La secuencia de agentes es:

HUMANO → Specification Agent → SPEC → Architecture Agent → ARQUITECTURA → Planning Agent → IMPLEMENTATION PLAN → Coding Agent → CÓDIGO → QA Agent → VERIFICACIÓN

Agente / etapa	Pregunta	Resultado
Humano / negocio	¿Qué problema existe?	Necesidad y contexto
Specification Agent	¿Qué debe hacer el sistema?	SPEC aprobada
Architecture Agent	¿Cómo se estructurará técnicamente?	Arquitectura + ADR
Planning Agent	¿Qué trabajo debe realizarse?	Plan de implementación
Coding Agent	¿Cómo se implementa?	Código
QA Agent	¿Cumple?	Pruebas y evidencia

3. Principio fundamental

IA propone → humano decide → equipo valida.

El Architecture Agent puede generar alternativas, comparar opciones y formular recomendaciones, pero no debe convertir automáticamente una propuesta de IA en una decisión arquitectónica. Las decisiones relevantes deben ser comprendidas, justificadas y aprobadas por el equipo.

4. Entrada: SPEC FROZEN

El punto de partida recomendado es una SPEC validada y congelada. La documentación de Specification Agent establece el flujo SPEC v0.x → preguntas pendientes → respuesta humana → SPEC v1.0 → revisión → APROBADA → SPEC FROZEN → Architecture Agent.

- Objetivo
- Alcance y fuera de alcance
- Actores
- Requisitos funcionales
- Requisitos no funcionales
- Reglas de negocio
- Flujos principales y alternativos
- Casos límite

- Criterios de aceptación
- Dependencias
- Restricciones
- Integraciones conocidas
- Decisiones aprobadas

5. Responsabilidad exacta del Architecture Agent

- Analizar la SPEC aprobada.
- Identificar los drivers arquitectónicos.
- Identificar restricciones arquitectónicas.
- Identificar atributos de calidad relevantes.
- Proponer alternativas arquitectónicas.
- Comparar alternativas mediante criterios explícitos.
- Recomendar una arquitectura adecuada al contexto.
- Documentar la decisión mediante ADR.
- Definir componentes, responsabilidades y fronteras.
- Definir integraciones y principales flujos técnicos.
- Identificar riesgos y supuestos técnicos.
- Preparar el Architecture Package para Planning.

6. Lo que NO debe hacer

- No debe redefinir silenciosamente el problema de negocio.
- No debe inventar requisitos.
- No debe modificar reglas de negocio sin aprobación.
- No debe seleccionar tecnología únicamente por preferencia o moda.
- No debe imponer microservicios, arquitectura hexagonal u otra arquitectura sin justificarla.
- No debe convertir la arquitectura directamente en código completo.
- No debe ocultar incertidumbres.
- No debe presentar como decisión humana una recomendación generada por IA.

7. Fase 1 — Identificación de Architecture Drivers

Un Architecture Driver es un requisito, restricción o atributo de calidad que tiene influencia significativa sobre la estructura de la solución.

Driver	Pregunta
Seguridad	¿Qué protección necesita el sistema?
Rendimiento	¿Qué tiempos de respuesta son relevantes?
Disponibilidad	¿Qué nivel de continuidad se requiere?
Escalabilidad	¿Qué crecimiento debe soportar?
Mantenibilidad	¿Qué tan fácil debe ser modificarlo?
Modificabilidad	¿Qué partes pueden cambiar frecuentemente?

Testabilidad	¿Qué tan fácil debe ser verificar componentes?
Interoperabilidad	¿Con qué sistemas debe integrarse?
Observabilidad	¿Cómo se detectarán y diagnosticarán problemas?
Fiabilidad	¿Qué comportamiento se espera ante fallos?

8. Fase 2 — Restricciones arquitectónicas

El agente debe distinguir los drivers de las restricciones.

Tipo	Ejemplo
Driver	Alta mantenibilidad
Driver	Integración con sistemas externos
Restricción	Infraestructura ya existente
Restricción	Tecnología exigida por el contexto
Restricción	Presupuesto disponible
Restricción	Tiempo de desarrollo
Restricción	API institucional obligatoria

Una restricción aprobada forma parte del contexto de decisión y no debe ser ignorada por el agente.

9. Fase 3 — Generación de alternativas

El Architecture Agent no debería saltar inmediatamente a una única solución.

- Arquitectura por capas
- Modular Monolith
- Service-Oriented Architecture
- Clean Architecture
- Hexagonal / Ports and Adapters
- Microservices
- Event-Driven Architecture
- API-First

Estas alternativas no son necesariamente mutuamente excluyentes. Una solución puede combinar decisiones, por ejemplo Modular Monolith con principios de Clean Architecture o Ports and Adapters, siempre que exista justificación.

10. Fase 4 — Comparación de alternativas

La comparación debe utilizar criterios relacionados con los drivers de arquitectura.

Criterio	Pregunta
Mantenibilidad	¿Qué tan fácil será modificar el sistema?
Escalabilidad	¿Cómo responde al crecimiento?
Complejidad	¿Qué complejidad adicional introduce?
Seguridad	¿Cómo favorece el control de seguridad?
Integración	¿Cómo resuelve las integraciones?

Tiempo	¿Es viable dentro del plazo?
Riesgo técnico	¿Qué incertidumbres introduce?
Testabilidad	¿Qué tan fácil será probarlo?
Observabilidad	¿Cómo se diagnosticará el comportamiento?

La matriz debe utilizar valores y pesos justificados por el contexto. Los números no deben inventarse para favorecer una arquitectura predeterminada.

11. Fase 5 — Selección de arquitectura

La selección debe explicar por qué una alternativa satisface mejor los requisitos y restricciones del proyecto. La pregunta no es "¿cuál arquitectura es la mejor?", sino "¿cuál es la más adecuada para este contexto?"

12. Fase 6 — Architecture Decision Record (ADR)

Elemento	**Contenido**
ID	Identificador de la decisión
Título	Nombre corto
Estado	Propuesta, aceptada, reemplazada, etc.
Contexto	Situación que origina la decisión
Problema	Qué debe resolverse
Alternativas	Opciones consideradas
Decisión	Opción seleccionada
Justificación	Por qué se seleccionó
Consecuencias positivas	Beneficios
Consecuencias negativas	Costos o compromisos
Supuestos	Condiciones asumidas
Evidencia	Información que respalda la decisión

13. Fase 7 — Definición de la estructura arquitectónica

La arquitectura debe expresar al menos, cuando aplique:

- Actores externos
- Sistemas externos
- Interfaces
- Componentes
- Módulos
- Servicios
- Persistencia
- Integraciones
- Flujos principales
- Límites de responsabilidad
- Dependencias

El diagrama debe comunicar la estructura y responsabilidades; no debe convertirse en un diagrama decorativo.

14. Arquitectura versus diseño

Nivel	Pregunta	Ejemplo de salida
Arquitectura	¿Cómo estructuramos el sistema?	Módulos, capas, límites, integraciones
Diseño	¿Cómo organizamos internamente una parte?	Interfaces, servicios, dependencias
Patrones	¿Qué solución conocida resuelve un problema de diseño?	Strategy, Adapter, Repository, etc.

El Architecture Agent debe mantener estas fronteras para evitar que una decisión de patrón aparezca prematuramente como una decisión de arquitectura.

15. Fase 8 — Patrones de diseño

Secuencia recomendada:

SPEC → Arquitectura → Problema de diseño → Alternativas → Patrón → Implementación → Validación

Problema	Patrón posible
Algoritmos intercambiables	Strategy
Interfaz externa incompatible	Adapter
Interfaz simplificada sobre subsistemas	Facade
Creación de objetos bajo distintas condiciones	Factory Method
Notificación de cambios	Observer
Secuencia de responsables	Chain of Responsibility
Persistencia separada del dominio	Repository
Transferencia de datos entre fronteras	DTO
Gestión explícita de dependencias	Dependency Injection

La selección es contextual. No se evalúa la cantidad de patrones, sino la pertinencia de la decisión.

16. Fase 9 — Riesgos, supuestos y preguntas técnicas

El Architecture Agent debe hacer explícito lo que todavía no está suficientemente determinado.

- Riesgos técnicos
- Supuestos
- Dependencias externas
- Decisiones pendientes
- Limitaciones conocidas
- Puntos que requieren validación

17. Fase 10 — Architecture Package

- Architecture Drivers
- Restricciones
- Atributos de calidad
- Alternativas consideradas
- Matriz de decisión
- Arquitectura seleccionada
- Diagramas
- Componentes y responsabilidades
- Integraciones
- ADR
- Decisiones de diseño
- Patrones justificados
- Riesgos técnicos
- Supuestos
- Preguntas técnicas abiertas
- Trazabilidad

18. Trazabilidad

La arquitectura debe poder relacionarse con los elementos que la originaron:

REQ → SPEC → Architecture Driver → Decisión Arquitectónica → Componente → Implementación → TEST

Esto permite explicar por qué existe cada decisión y posteriormente verificar su cumplimiento.

19. Uso responsable de IA

IA puede	**Equipo debe**
Proponer arquitecturas	Evaluar alternativas
Generar matrices preliminares	Validar criterios y pesos
Proponer componentes	Confirmar responsabilidades
Sugerir patrones	Verificar que exista un problema real
Generar diagramas preliminares	Revisar coherencia
Redactar ADR	Validar decisión y consecuencias
Detectar riesgos posibles	Confirmar si aplican

La IA no sustituye el razonamiento arquitectónico.

20. Prompt base para Architecture Agent

Actúe como Architecture Agent dentro de un proceso de Spec-Driven Development.

Entrada:
- SPEC aprobada y congelada.
- Restricciones conocidas.

- Contexto técnico disponible.
- Convenciones del proyecto.

Objetivo:
Diseñar y justificar una arquitectura técnica que satisfaga los requisitos y atributos de calidad definidos en la SPEC.

Reglas:
1. No invente requisitos ni reglas de negocio.
2. Distinga hechos, supuestos, propuestas y decisiones aprobadas.
3. Identifique primero los Architecture Drivers.
4. Identifique restricciones.
5. Proponga alternativas antes de recomendar una.
6. Compare las alternativas con criterios explícitos.
7. Justifique la arquitectura seleccionada.
8. Documente las decisiones mediante ADR.
9. Separe arquitectura, diseño y patrones.
10. No genere código de producción como sustituto de la arquitectura.
11. Señale preguntas técnicas abiertas.
12. Toda decisión relevante debe poder ser explicada y defendida.

Salida:
1. Architecture Drivers.
2. Restricciones.
3. Atributos de calidad.
4. Alternativas.
5. Matriz de decisión.
6. Arquitectura recomendada.
7. Diagrama conceptual.
8. Componentes y responsabilidades.
9. Integraciones.
10. ADR.
11. Decisiones de diseño.
12. Patrones potenciales y justificación.
13. Riesgos.
14. Supuestos.
15. Preguntas abiertas.
16. Trazabilidad.

21. Ejemplo de razonamiento

Necesidad especificada: el sistema debe calcular el costo de un domicilio según la distancia.

Specification Agent: define el comportamiento requerido, sin seleccionar tecnologías ni patrones.

Architecture Agent: analiza dónde debe residir la capacidad de cálculo y cómo aislarla de otras partes.

Diseño: identifica si existen algoritmos intercambiables.

Patrón: Strategy puede ser considerado si realmente existen algoritmos intercambiables.

Conclusión: el patrón se deriva del problema de diseño; no debe imponerse desde la SPEC.

22. Criterios para evaluar al Architecture Agent

- ¿La arquitectura responde a la SPEC?
- ¿Los principales drivers están identificados?
- ¿Las restricciones fueron consideradas?
- ¿Se compararon alternativas?
- ¿La decisión está justificada?
- ¿Los atributos de calidad influyeron realmente?
- ¿Las responsabilidades están claramente separadas?
- ¿Los componentes y fronteras son coherentes?
- ¿Los patrones utilizados tienen un problema que resolver?
- ¿Los riesgos y supuestos están explícitos?
- ¿La arquitectura puede ser defendida por el equipo sin depender de la IA?
- ¿Existe trazabilidad entre requisitos y decisiones?

23. Preguntas para la defensa técnica

1. ¿Qué requisito de la SPEC condicionó más la arquitectura?
2. ¿Cuáles fueron los principales Architecture Drivers?
3. ¿Qué alternativas consideraron?
4. ¿Por qué descartaron las otras alternativas?
5. ¿Qué atributo de calidad tuvo mayor peso?
6. ¿Qué trade-off aceptaron?
7. ¿Qué decisión está documentada en cada ADR?
8. ¿Por qué utilizaron esta arquitectura y no microservicios?
9. ¿Por qué utilizaron o descartaron arquitectura hexagonal?
10. ¿Qué problema concreto justificó cada patrón?
11. ¿Qué recomendación de IA rechazaron y por qué?
12. ¿Qué cambiaría en la arquitectura si aumentara significativamente la carga?
13. ¿Dónde están los principales riesgos técnicos?

24. Flujo resumido

SPEC FROZEN → Architecture Drivers → Restricciones → Atributos de calidad → Alternativas → Comparación → Decisión → ADR → Arquitectura → Diseño → Patrones → Architecture Package → Planning Agent

25. Regla de oro

El Architecture Agent no debe buscar la arquitectura más moderna ni la más compleja. Debe buscar la arquitectura más adecuada para satisfacer la SPEC, sus atributos de calidad y sus restricciones, y debe dejar evidencia suficiente para que el equipo pueda explicar y defender cada decisión.

26. Base documental

Esta guía se construye a partir de la documentación de trabajo sobre Specification Agent proporcionada para el proyecto. Dicha documentación establece la relación entre SPEC, Architecture Agent, Planning Agent, Coding Agent y QA Agent; la necesidad de mantener el control humano; la trazabilidad desde requisitos hasta pruebas; y el límite entre especificación y decisiones técnicas.

Referencia normativa mencionada en la documentación base: ISO/IEC/IEEE 29148:2018, Requirements engineering.

Referencia normativa y recursos de consulta

ISO/IEC/IEEE 29148:2018
Systems and software engineering — Life cycle processes — Requirements engineering

Propósito de esta referencia

Este documento se incorpora como material de referencia para la guía del **Specification Agent** dentro del enfoque de Spec-Driven Development. Su propósito es proporcionar al estudiante una ruta de consulta oficial y legal sobre la norma ISO/IEC/IEEE 29148:2018, sin reproducir el texto protegido por derechos de autor.

Referencia normativa

ISO/IEC/IEEE 29148:2018. *Systems and software engineering — Life cycle processes — Requirements engineering*. Edition 2, 2018. ISO/IEC/IEEE.

La página oficial de ISO indica que esta edición fue publicada en noviembre de 2018, contiene 92 páginas y fue revisada y confirmada en 2024; por tanto, la edición 2018 continúa siendo la versión vigente mientras se desarrolla su reemplazo.

Consulta gratuita oficial

ISO dispone de una **Online Browsing Platform (OBP)** que permite consultar gratuitamente en línea contenido de la norma, incluido su índice, alcance, referencias normativas, conceptos, procesos e información relacionada con ingeniería de requisitos.

Recurso	Enlace
ISO — ficha oficial de ISO/IEC/IEEE 29148:2018	https://www.iso.org/standard/72089.html
ISO — Online Browsing Platform (consulta gratuita)	https://www.iso.org/obp/ui?_escaped_fragment_=iso%3Astd%3Aiso-iec-ieee%3A29148%3Aed-2%3Av1%3Aen
ISO — página en español de la norma	https://www.iso.org/cms/render/live/es/sites/isoorg/contents/data/standard/07/20/72089.html

Contenido que puede consultarse gratuitamente

• Alcance (Scope).

• Referencias normativas.

• Términos, definiciones y abreviaturas.

• Fundamentos de los requisitos.

• Consideraciones prácticas.

• Información de requisitos.

• Procesos de requisitos.

• Análisis de negocio o misión.

• Definición de necesidades y requisitos de las partes interesadas.

• Definición de requisitos del sistema/software.

• Gestión de requisitos.

• Información y estructura de los productos documentales relacionados con requisitos.

Relación con el Specification Agent

La norma constituye una referencia para fundamentar el proceso de ingeniería de requisitos utilizado en el taller. En particular, sirve como marco para trabajar la identificación y análisis de necesidades, definición de requisitos, gestión de requisitos e información asociada.

La guía del Specification Agent no constituye una reproducción ni una implementación normativa completa de ISO/IEC/IEEE 29148:2018. Es un recurso pedagógico adaptado al proceso de Spec-Driven Development utilizado en el curso.

Referencias normativas relacionadas

La propia ISO/IEC/IEEE 29148:2018 identifica como referencias normativas ISO/IEC/IEEE 15288:2015, *Systems and software engineering — System life cycle processes*, e ISO/IEC/IEEE 12207:2017, *Systems and software engineering — Software life cycle processes*.

Norma	Consulta oficial
ISO/IEC/IEEE 15288	https://www.iso.org/standard/63711.html
ISO/IEC/IEEE 12207	https://www.iso.org/standard/63712.html

Estado actual de la norma

A la fecha de elaboración de este material, ISO informa que ISO/IEC/IEEE 29148:2018 continúa vigente, pero se encuentra en proceso de revisión. ISO/IEC/IEEE DIS 29148 corresponde al borrador de la tercera edición y reemplazará a la edición 2018 cuando el proceso de normalización concluya.

Consulta oficial del proyecto de nueva edición: https://www.iso.org/standard/94091.html

Nota sobre acceso y derechos de autor

El texto completo de ISO/IEC/IEEE 29148:2018 no se encuentra disponible gratuitamente como descarga oficial. La norma puede adquirirse a través de ISO/IEC/IEEE o consultarse mediante mecanismos institucionales que dispongan de la licencia correspondiente. La OBP proporciona una consulta web de contenido de la norma y constituye la opción oficial gratuita recomendada para los estudiantes.

Fuente principal verificada: International Organization for Standardization (ISO), ISO/IEC/IEEE 29148:2018.